Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра компьютерных систем в управлении и проектировании (КСУП)

СВЯЗНЫЙ СПИСОК

Отчет по лабораторной работе №2
по дисциплине «Структуры данных»

Студент гр. 575-3
_____ И.Н. Перевозчиков
20 октября 2026 г.

Руководитель
кафедры КСУП
____ _____ Н.А. Набережнев
___
оценка 22 октября 2026 г.

Томск 2026

Оглавление

Введение..... 3

1 теоретическая часть. 4

2 методика исследования..... 5

Заключение 13

Введение

Цель работы – сравнительный анализ производительности двух динамических структур данных: динамического массива и двусвязного списка. В ходе работы необходимо реализовать указанные структуры, измерить время выполнения базовых операций (вставка, удаление) и на практике подтвердить теоретические оценки алгоритмической сложности.

Время выполнения операций над структурами данных зависит от двух основных факторов: размера структуры (количество элементов N) и места операции (начало, конец или середина).

В динамическом массиве элементы хранятся в непрерывном блоке памяти, что обеспечивает быстрый доступ по индексу за O(1), но вставка или удаление элемента в середине или начале требует сдвига всех последующих элементов, что занимает время O(N).

В двусвязном списке элементы (узлы) разбросаны по памяти и связаны указателями. Доступ к элементу по индексу требует последовательного перебора (O(N)), однако сами операции вставки и удаления (если известен адрес узла) выполняются за константное время O(1), без необходимости сдвигать другие элементы.

Таким образом, возникает проблема выбора оптимальной структуры в зависимости от преобладающих операций в конкретной задаче.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.

Для оценки эффективности алгоритмов используется асимптотическая сложность в нотации «О-большое» (Big-O). Она показывает, как время выполнения операции растет при увеличении количества элементов N в структуре.

Таблица 1 – Теоретическая оценка сложности операций для структур

Операция	**Динамический массив**	**Двусвязный список**
Вставка в начало	O(N)	O(1)
Вставка в конец	O(1)*	O(1)
Вставка в середину	O(N)	O(N)(поиск)/O(1)(вставка)
Удаление из начала	O(N)	O(1)
Удаление из конца	O(1)	O(1)
Удаление из середины	O(N)	O(N)(поиск)/O(1)(удаление)

2 МЕТОДИКА ИССЛЕДОВАНИЯ

Описание реализованных структур данных:

Динамический массив (структура DynamicArray): Представляет собой структуру, хранящую указатель на непрерывный блок памяти (int* _array), текущий размер (_size) и выделенную вместимость (_capacity). При добавлении элементов, если _size превышает _capacity, массив перевыделяется (вместимость увеличивается в 2 раза).

Двусвязный список (структура List): Состоит из узлов (структура Node), каждый из которых хранит данные, указатель на следующий (_nextNode) и предыдущий (_previousNode) узлы. Сама структура List хранит указатели на начало (_head), конец (_tail) и количество элементов (_size).

Обоснование выбора измеряемых операций:

Для тестирования выбраны операции вставки (AddElement / AddNode) и удаления (RemoveByIndex / RemoveNodeByIndex) из начала, конца и середины структуры.

Операции в начале и в конце демонстрируют различия в работе с граничными элементами (сдвиг всего массива против перестановки пары указателей).

Операции в середине показывают накладные расходы на поиск элемента (для списка) и сдвиг половины элементов (для массива).

Измерение времени производится с использованием библиотеки <chrono> (класс std::chrono::high_resolution_clock). Так как время выполнения одной операции может быть меньше точности таймера, используется метод усреднения: тестируемая операция выполняется в цикле M раз (например, 10 000 итераций), после чего суммарное время делится на количество итераций. Перед замерами выполняется «холостой» прогон для прогрева кэша процессора.

Замеры проводились для структур размером N=1000, 10000 и 100000 элементов.

3 ОСНОВНАЯ ЧАСТЬ

1. Вставка в начало (Insert at beginning)

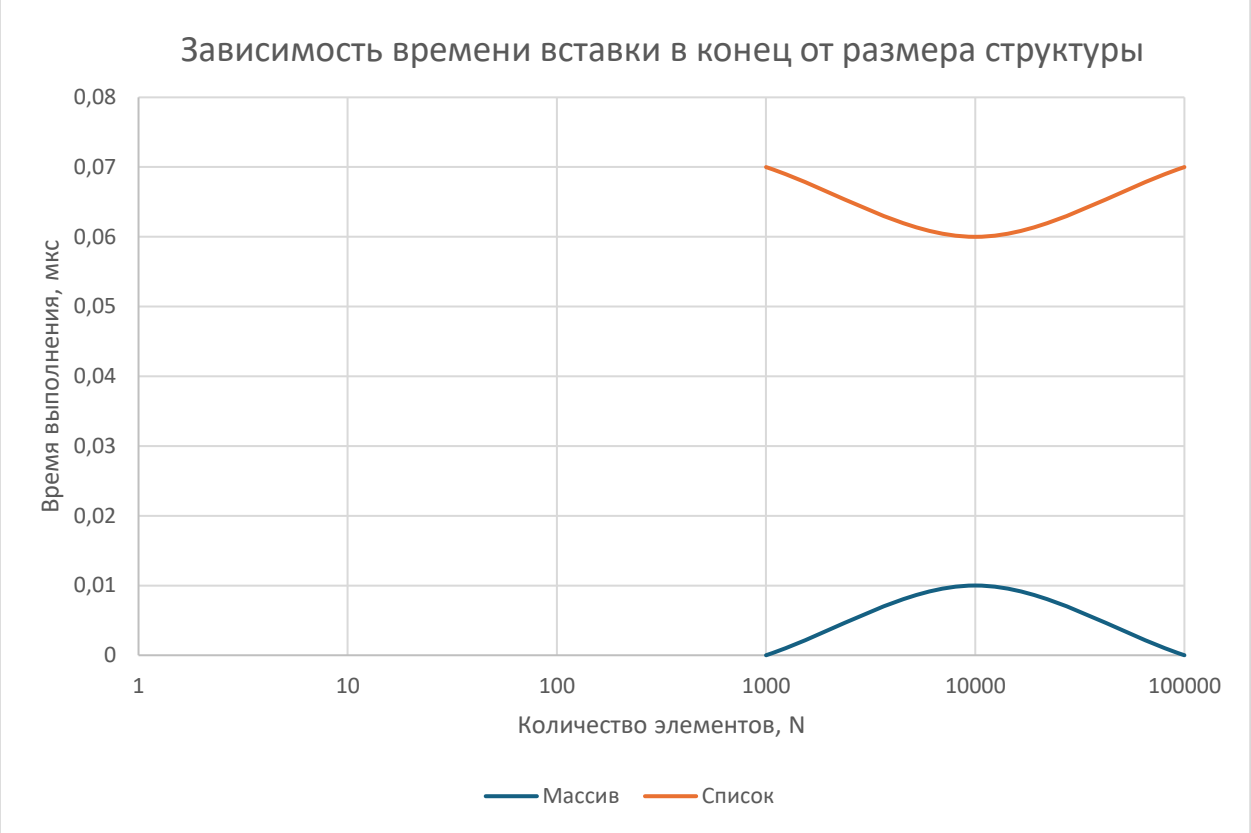
N	Массив	Список
1000	3,26	0,07
10000	32,29	0,08
100000	279,15	0,07



График для массива демонстрирует ярко выраженный линейный рост: при увеличении N в 10 раз (с 1000 до 10000) время растет с 3.260 до 32.290 мкс (примерно в 10 раз), а при 100000 достигает 279.150 мкс. Это объясняется тем, что при вставке в начало массива требуется сдвинуть все существующие N элементов на одну позицию вправо, что занимает O(N) времени. График для списка представляет собой практически горизонтальную прямую (0.070 мкс на всех размерах). Это подтверждает оценку O(1): двусвязному списку достаточно выделить новый узел и переставить несколько указателей (`_head` и `next` предыдущего), независимо от общего количества элементов.

2. Вставка в конец (Insert at end)

N	Массив	Список
1000	0	0,07
10000	0,01	0,06
100000	0	0,07



Обе кривые на графике лежат почти на нуле. Для массива время составляет 0.000–0.010 мкс, для списка — 0.060-0.070 мкс. Для массива это O(1), так как в бенчмарке заранее выделена избыточная емкость (capacity = 3n), и операция сводится к записи элемента в конец и инкременту размера без перевыделения памяти (Resize). Для списка это также O(1), так как структура хранит указатель на хвост (_tail), и вставка не требует обхода элементов.

3. Вставка в середину (Insert in middle)

N	Массив	Список
1000	1,49	1,38
10000	15,36	33
100000	148,65	376,02



Оба графика демонстрируют линейный рост, но с разной крутизной. Для массива время растет с 1.490 до 148.650 мкс, для списка — с 1.380 до 376.020 мкс. Для массива это O(N), поскольку для вставки в середину необходимо сдвинуть половину элементов (N/2) вправо. Для списка это тоже O(N), но по другой причине: чтобы вставить элемент, нужно сначала найти позицию вставки, а метод `GetNodeByIndex` вынужден последовательно перебирать узлы с начала. При N=100000 список оказывается медленнее массива из-за плохой локальности памяти (узлы разбросаны по куче, что вызывает частые кэш-промахи).

4. Удаление из начала (Remove from beginning)

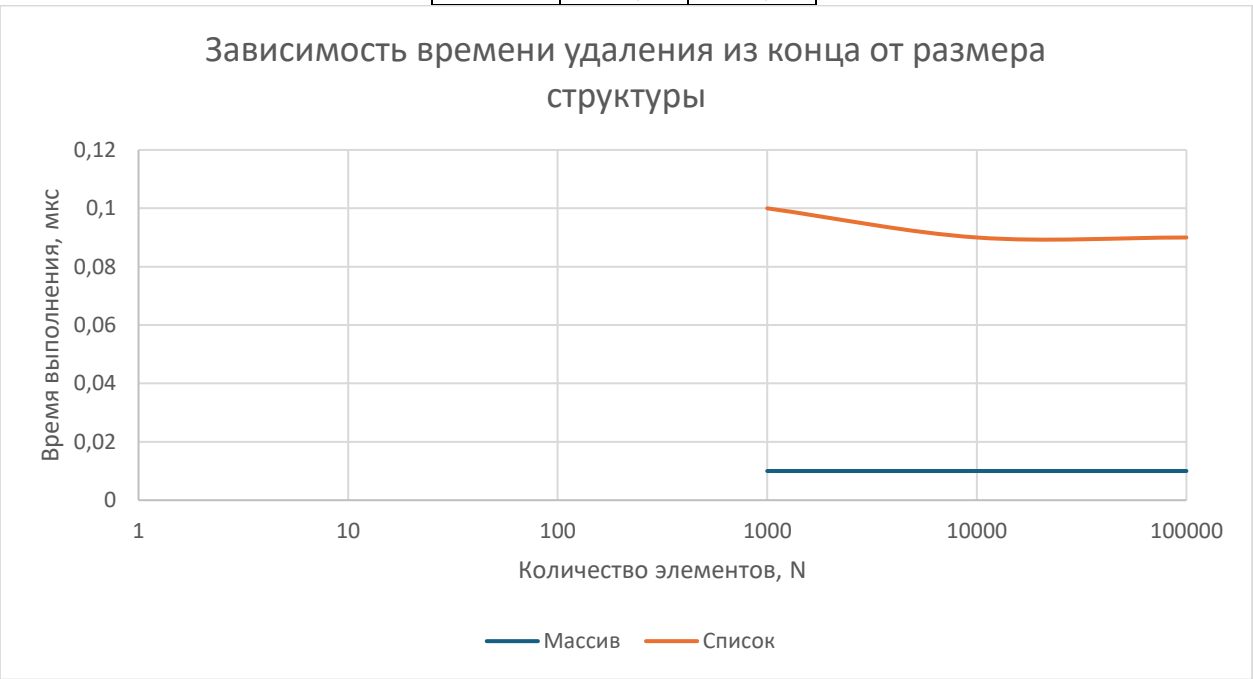
N	Массив	Список
1000	3,04	0,09
10000	30,97	0,08
100000	283,1	0,08



График для массива показывает линейный рост (3.040 -> 30.970 -> 283.100 мкс). При удалении первого элемента необходимо сдвинуть все оставшиеся N-1 элементов влево, чтобы закрыть образовавшуюся «дыру». Это требует O(N) операций. График для списка остается плоским (0.080–0.090 мкс). Удаление первого узла — это O(1), так как достаточно просто переставить указатель `_head` на следующий узел и удалить старый.

5. Удаление из конца (Remove from end)

N	Массив	Список
1000	0,01	0,1
10000	0,01	0,09
100000	0,01	0,09



Обе кривые на графике горизонтальны. Для массива время составляет 0.010 мкс, для списка — 0.090 мкс. Для массива это O(1), так как удаление последнего элемента не требует сдвига (цикл сдвига не выполняется), достаточно просто уменьшить счетчик `_size`. Выделенного запаса памяти хватает, чтобы не вызывать сжатие (Shrink). Для списка это также O(1) благодаря наличию указателя `_tail` (хвост). Мы сразу получаем доступ к последнему элементу и его предыдущему узлу, не обходя весь список.

6. Удаление из середины (Remove from middle)

N	Массив	Список
1000	1,54	1,18
10000	18,22	29,48
100000	143,25	654,23



Оба графика имеют линейный вид. Время массива растет с 1.540 до 143.250 мкс, списка — с 1.180 до 654.230 мкс. Для массива это O(N) из-за необходимости сдвигать правую половину элементов влево после удаления. Для списка это O(N) из-за необходимости поиска удаляемого элемента с помощью GetNodeByIndex (проход по указателям).

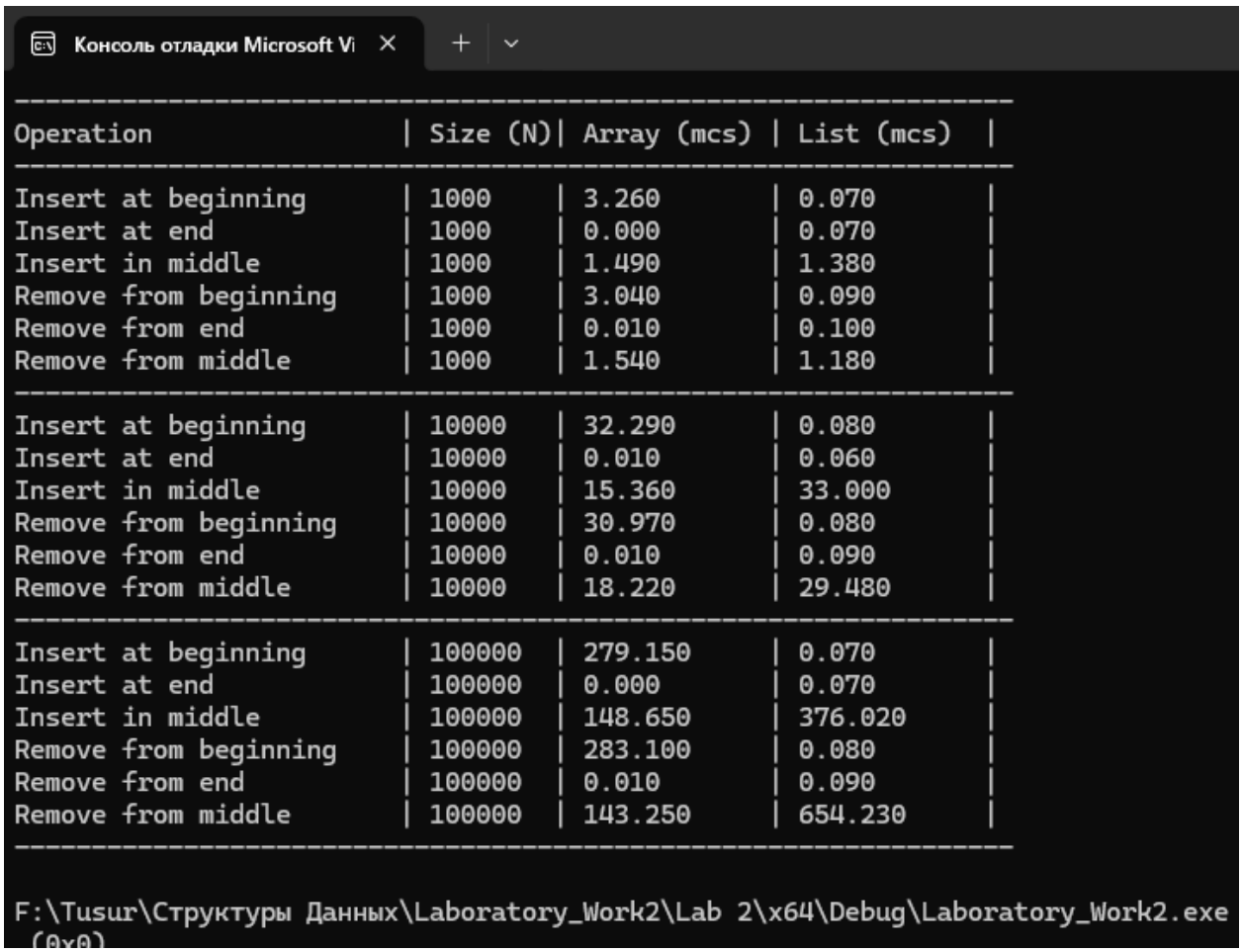


Рисунок 1 – Результаты тестирования производительности

Заключение

В ходе лабораторной работы были исследованы операции вставки и удаления для динамического массива и двусвязного списка.

Динамический массив показывает высокую скорость O(1) на операциях с концом, но деградирует до O(N) при операциях в начале или середине из-за необходимости сдвига элементов. Двусвязный список демонстрирует стабильную скорость O(1) на концах, но требует O(N) времени для доступа к произвольному элементу (вставка/удаление в середине).